Test-Time Adaptive Anomaly Detection for Sensor-Based Time-Series Monitoring Under Distribution Shift

Group No : 2
Guided By : Dr. Arpita Mandal
Phase 1 recap and Phase 2 progress

Team Members

Sl.No	PRN	Name
1	SCFU424005	Tanishka Mahindrakar
2	SCFU424023	Unnati Patro
3	SCFU424012	Purav Patil

Agenda

- Problem statement
- Objectives
- Problem Description
- Proposed Solution
- Dataset
- Training/ Pre-requisites
- Timeline/Milestones
- References

Domain and Problem Statement

- Develop a test-time adaptation module for an unsupervised time-series anomaly detector on the Yahoo Webscope dataset to adapt to changes in normal time-series behavior during testing, reduce false alarms caused by distribution shifts, and improve detection performance measured using F1 score and AUROC compared with a static (non-adaptive) baseline detector.

Objectives

- Implement an EMA-based trend estimation and detrending module for changing data behavior.
- Implement test-time model adaptation using samples predicted as normal.
- Compare the adaptive model with a static baseline using F1, AUROC and AUPRC.

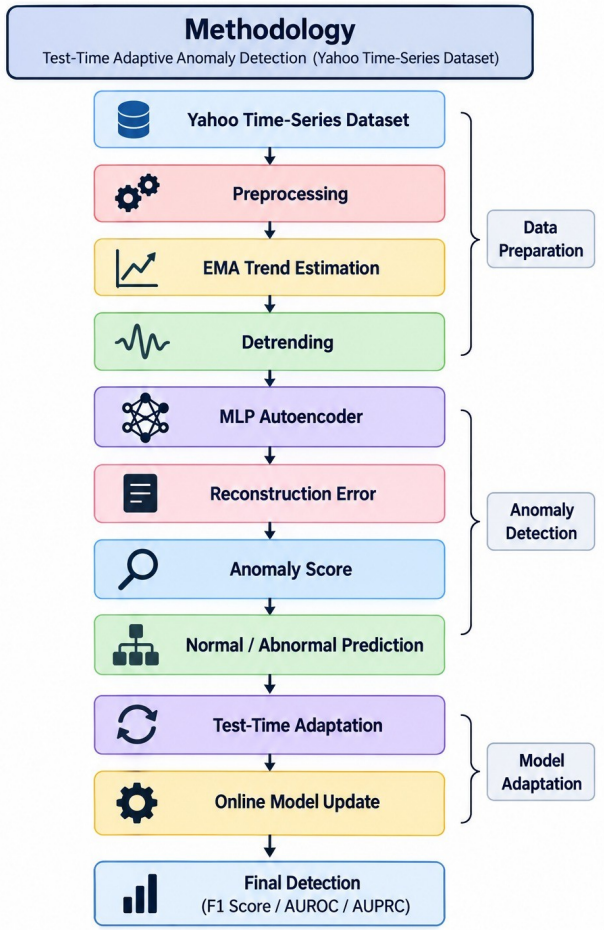
Dataset

- There are 9 Datasets in the Paper
- We have chose **Yahoo Webscope S5 dataset**

Methods & Proposed Solution

- The proposed method uses the Yahoo time-series dataset, followed by preprocessing and EMA-based trend estimation to remove changing trends.
- The detrended data is passed through an MLP Autoencoder to calculate reconstruction error and anomaly scores.
- During testing, the model adapts online using samples predicted as normal, followed by final anomaly detection and evaluation using F1 Score, AUROC, and AUPRC.

- Proposed Solution:
- An adaptive anomaly detection framework that combines EMA-based detrending with an MLP Autoencoder and test-time model adaptation.
- Evaluation: F1 Score | AUROC | AUPRC



Why EMA for Trend Estimation?

1)**Cheap & fast:** EMA is just one simple formula — no extra training, no heavy computation. Perfect for real-time systems where data keeps streaming in.

2)**No memory needed:** It only needs the *previous* trend value + current window average — doesn't need to store a long history. Matches their goal of "process and evict" (lightweight, real-time-friendly).

3)**Naturally adapts:** EMA automatically shifts toward new patterns while still being stable against noise — exactly what's needed to track a "moving normal.”

4)**Proven & simple:** It's a well-established statistical technique (from Muth, 1960) — reliable, not experimental.

Why MLP Autoencoder model?

1)**Their method is model-agnostic:** The paper's real contribution is the *adaptation strategy* (trend + online update), not a new architecture. So they picked a **simple, well-known baseline** (MLP autoencoder) to prove their strategy works — even on a basic model.

2)**If it works on a simple model, it's a strong result:** Showing improvement on a "plain" MLP proves the gain comes from their trend/adaptation idea — not from a fancy, complicated model doing the heavy lifting.

3)**Reconstruction-based fits naturally:** Autoencoders are built to reconstruct normal data well and struggle with anomalies — this fits perfectly with their loss function (Eq. 4), which is based on reconstruction error.

4)**Easy to plug their method into:** Their normalize → model → denormalize pipeline (Fig. 3) is architecture-agnostic — it just needed *some* reconstruction model to demonstrate the idea, and MLP is the simplest choice.

Outcomes

- Our project mainly focuses on:
  - Adapting anomaly detection models to changing normal time-series behavior.
  - Reducing false alarms caused by distribution shift during deployment.
  - Improving anomaly detection performance using test-time adaptation.
  - Comparing the adaptive model with a static baseline using F1 Score, AUROC, and AUPRC.

Progress at Phase 1

- Project Topic Finalised
- Dataset Identified
- Literature Study Initiated
- GitHub Repo Created
- Implementation Yet to Begin
- Gap analysis

Limitation

1. Threshold Problem

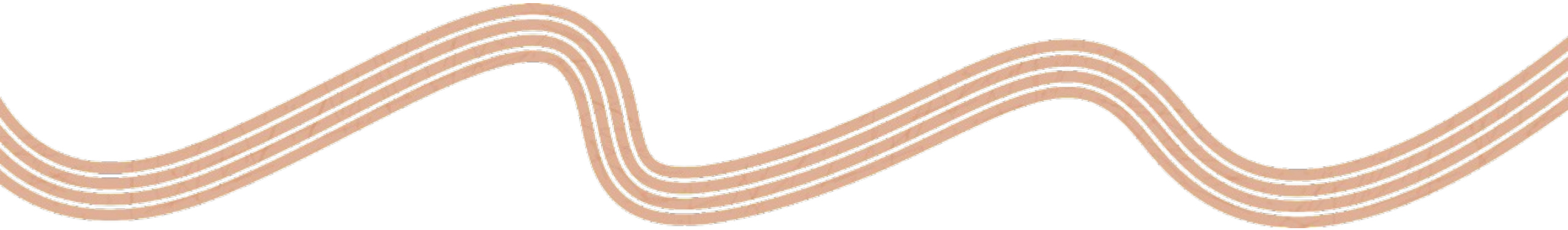
- Ideally, you'd pick the "anomaly cutoff" (threshold) by testing all possible values on the test data and choosing whichever works best.
- Problem: You can't do that in real life — you don't have test data with answers ahead of time.
- Their fix: They didn't use that "cheat" — instead, they set the threshold using only training data stats, and reported **AUROC** (a threshold-free performance score) to be fair.
- **Limitation:** Performance could be even better with a smarter threshold-picking method — they admit this is still an open problem.

2. Inconsistent Labeling (What counts as "anomaly" is unclear)

- Different situations define "anomaly" differently — there's no universal rule.
- Problem: When test data looks very different from training data, you can't always tell — is this a real anomaly, or just a legitimate "new normal" (distribution shift)?
- Their fix: They assume most test data is still normal, and use trend estimation + the model's own predictions to guess what's normal.
- **Limitation:** This is a workaround, not a real solution. A better fix could be **active learning** — having a human occasionally label a few test samples to guide the model, but they didn't do this; they just suggest it as future work.

*** Future Direction (mentioned, not implemented): Use active learning — have humans label a small subset of test data to reduce the ambiguity between "anomaly" vs. "new normal."**

Phase 2: Progress Update



Phase 2 Evaluation Agenda

1. High-Level Design
2. Model Development and Integration
3. Model Version, Metadata and Metrics
4. Git and GitHub
5. Folder and File Structure
6. Results and Next Steps

1. High-Level Design

How the system works, step by step

1. Server data
SMD: 38 sensors per machine

2. Windows
Cut the stream into short pieces

3. Normalise
Remove slow drift (EMA)

4. MLP autoencoder
Learns what normal looks like

5. Anomaly score
Copy error: big = unusual

6. Threshold
Above cut-off = alarm

7. Evaluate
F1, AUPRC, AUROC

Before deployment
The model learns from normal history only. No anomaly labels needed.

Test-time adaptation
While running, windows judged normal update the model.

Every model gets the same data, the same metrics and the same random seeds.

High-Level Design: Models Compared

Model	How it works	What it updates
Pretrained baseline	Trained once on normal data, then frozen	Nothing
M2N2 (AAAI 2024)	EMA detrending + update on windows judged normal	Whole MLP
CANDI (AAAI 2026)	Picks safe windows to learn from; trains a small add-on (SANA)	SANA add-on only

All three use the same MLP autoencoder, so differences come from the adaptation strategy.

High-Level Design: Architecture

Inputs
- SMD data
- Config files (YAML)

Evaluation harness
- Methods: M2N2, CANDI, baseline
- Thresholds: fixed, streaming
- Normalisation: EMA, RevIN
- Backbone: MLP (swappable)

Outputs
- Raw scores and alarms
- Metrics (12 per run)
- Run settings
- Comparison tables

Every method has the same two functions: fit() and score().

Thresholds and metrics are separate from the models, so all models are graded the same way.

2. Model Development: What Is Built

Component	Status
MLP autoencoder	✓ Done
Pretrained baseline (static)	✓ Done
M2N2 (adaptive method 1)	✓ Done
CANDI (adaptive method 2)	✓ Done
Metrics and tests	✓ Done
Streaming threshold	Coded, results next
RevIN normalisation	Coded, results next
Other backbones and latency	Planned

Done = built and run end to end on SMD data.

Model Integration

- **One config file** drives the whole run.
- **Steps:** load data, train, stream the test data and adapt, apply threshold, compute metrics, save.
- **Docker** packages the environment, so it runs the same on any machine.
- **54 full runs** (6 machines x 3 models x 3 seeds) and 6 quick runs completed.
- **22+ automated tests** cover metrics, thresholds, windows and normalisation.

Run it

docker compose run --rm data | quick | smd | compare

3. Model Version and Metadata

Model	Config file	Window	Test-time update	Parameters
Pretrained	configs/methods/pretrained.yaml	10	None (frozen)	≈ 102 K
M2N2	configs/methods/m2n2.yaml	5	SGD, 1 step per window	≈ 53 K
CANDI	configs/methods/candi.yaml	10	SGD on selected windows	≈ 102 K + 2.1 M

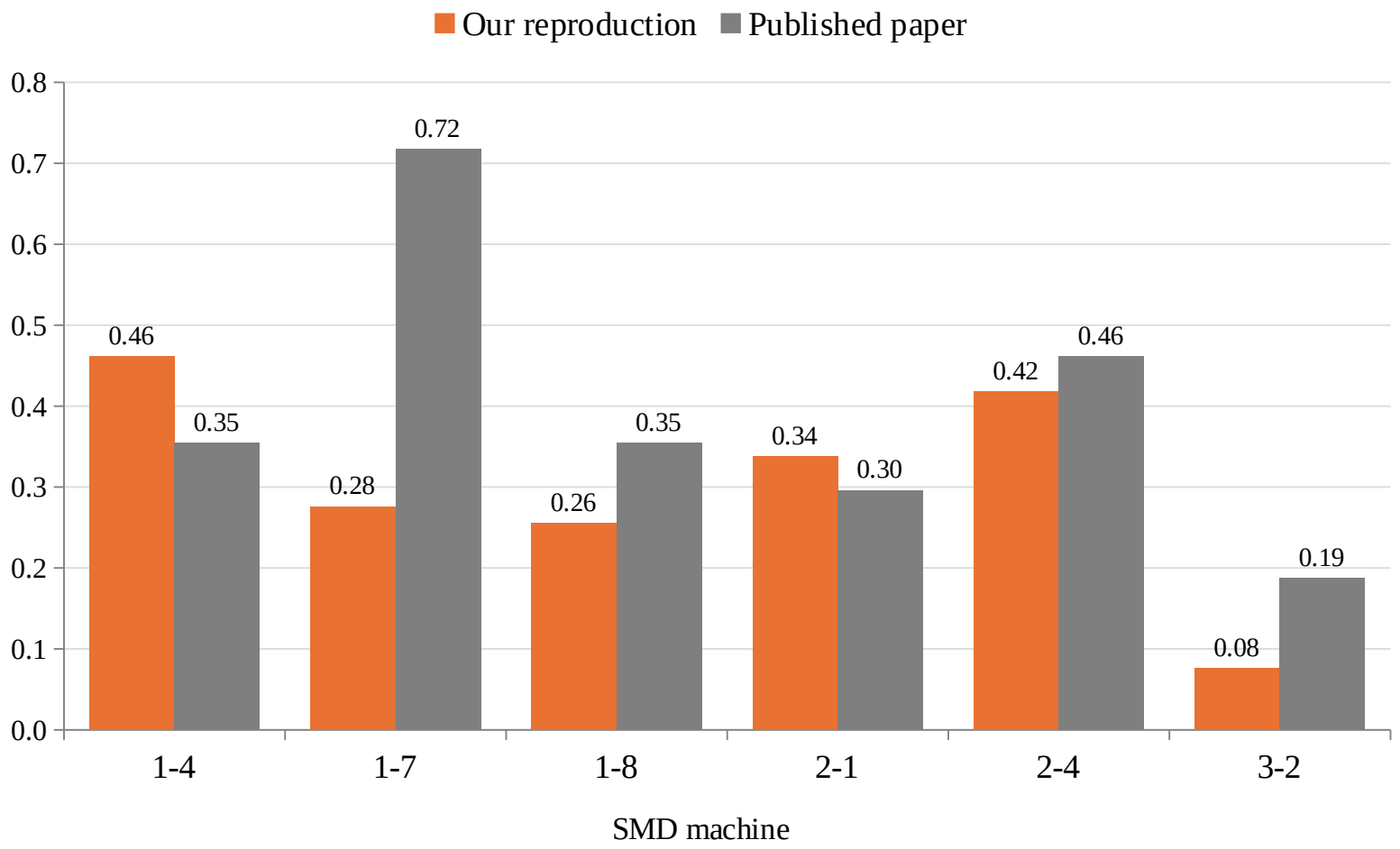
Saved automatically with every run

- run.json: settings, dataset, seed, Git commit, threshold, number of updates
- metrics.json: all 12 metric values
- scores.npy and preds.npy: raw score and alarm for every time step

Metrics Used

Metric	Meaning in simple words
F1 score	Balance of real alarms and caught anomalies (needs a threshold)
AUPRC	How well the scores rank the rare anomalies (no threshold needed)
AUROC	Chance an anomaly scores higher than normal data (no threshold needed)
PA%K, F1-PA	Credit for detecting part of an anomaly period (reference only)

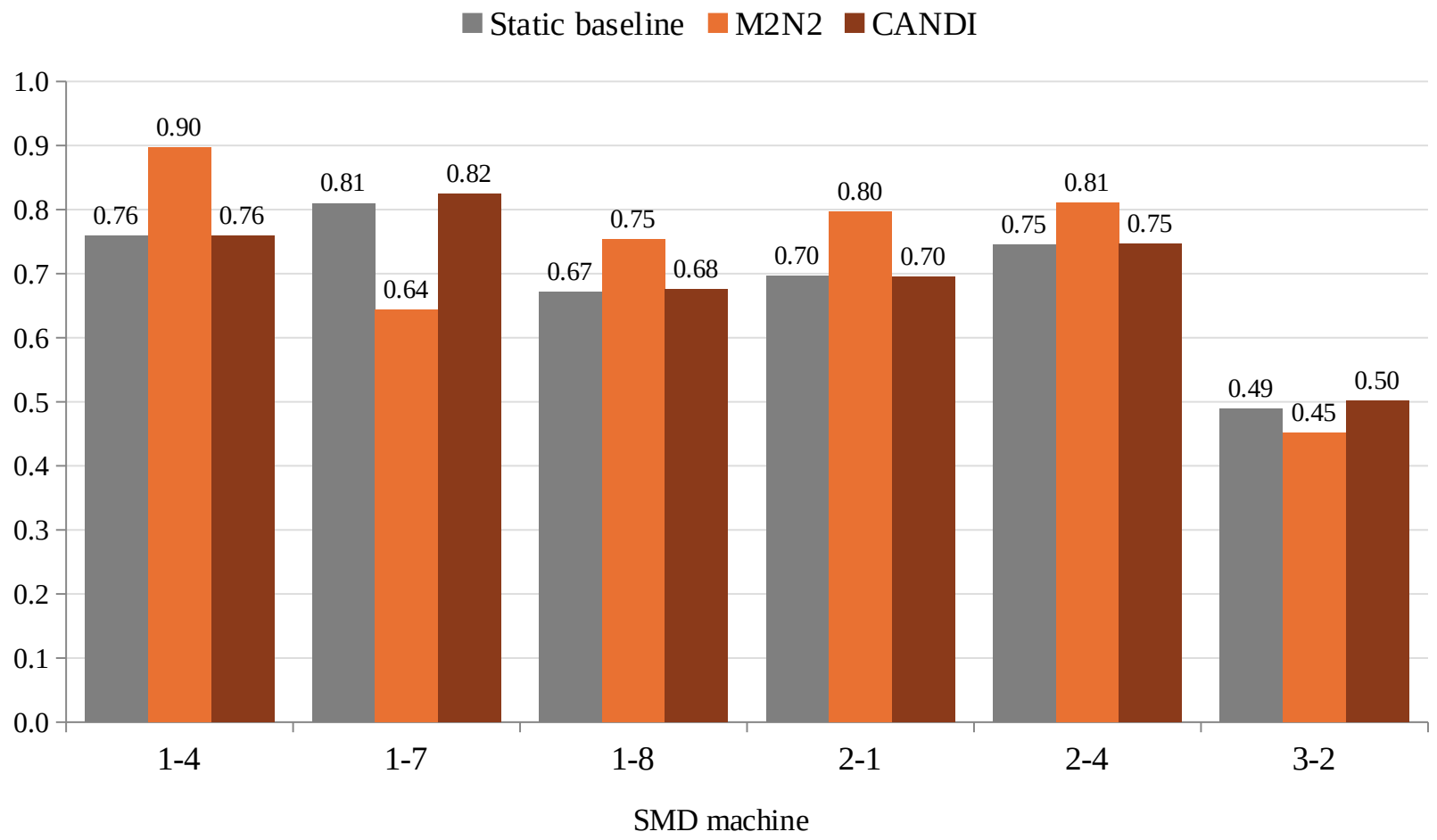
Results: Our M2N2 vs Published Paper (AUPRC)



Reading the chart

- Machines 1-4 and 2-1 (tuned in the paper): we match or beat it. AUROC 0.897 vs 0.845 and 0.797 vs 0.764.
- Machines 1-8 and 2-4: close.
- Machines 1-7 and 3-2: gap. We used default settings there.

Results: Static vs Adaptive Models (AUROC)



Mean AUROC

Static baseline: 0.695

M2N2: 0.726

CANDI: 0.700

M2N2 beats the static baseline on 4 of 6 machines. CANDI's gain is small so far; tuning it is our next step.

4. Git and GitHub

- **Repository:** github.com/PuravPatil/test-time-adaptation-tsad
- **Branch per person** (name/topic), merged by pull request with one review.
- **Automated tests** run on every pull request (GitHub Actions).
- **Original paper code** is never edited; our changes live in our own adapters.
- **Data and run outputs** are kept out of Git.

5. Folder and File Structure

Folder	Purpose
configs/	All settings: datasets, methods, experiments
src/audit/data, backbones	SMD loader, windowing, MLP autoencoder
src/audit/methods	M2N2 and CANDI behind one interface
src/audit/normalization, thresholds	EMA / RevIN; fixed / streaming threshold
src/audit/evaluation	Metrics, harness, latency timer
scripts/	Download, preprocess, run, compare with paper
tests/	Automated tests
results/, data/	Run outputs and summary tables; datasets
docs/, README.md, Dockerfile	Notes, how to run, container setup

Next Steps

- **Weeks 4-6:** three extension studies on the same harness.
  - **Tanishka:** RevIN vs EMA normalisation
  - **Purav:** streaming threshold vs fixed threshold
  - **Unnati:** other backbones and per-step latency
- **Tuning:** adjust M2N2 and CANDI settings to move closer to the papers.
- **Weeks 7-8:** live demo dashboard.

Thank you

